

Projet de traitement d'images : segmentation de gliomes sur des IRM cérébrales 2D et 3D à l'aide du deep learning

Rapport court

Table des matières

1	Introduction	3
1.1	Présentation du Dataset	3
1.2	Création du dataset 2D	4
2	IRM 2D	5
2.1	Pré-traitement des données	5
2.2	Modèles	5
2.2.1	U-Net	5
2.2.2	Attention U-Net	7
2.2.3	U-Net++	7
2.3	Métriques d'entraînement	8
2.4	Métriques de test	9
2.4.1	Mesures de performance	9
	Intersection over Union (IoU)	9
	Score de Dice	9
2.4.2	Matrice de confusion	10
2.4.3	Affichage de la prédiction	11
2.5	Résultats	12
2.5.1	Comparaison de l'entraînement avec une seule et quatre modalités	12
3	IRM 3D	13
4	Bibliographie	14

Les gliomes (ou tumeurs gliales) sont des tumeurs cérébrales naissant dans les cellules gliales, cellules dont le rôle est de soutenir et protéger les neurones.

Le but du projet est de tester différentes techniques de deep learning pour la détection (segmentation) supervisée de gliomes sur des IRM. Le langage utilisé est Python, et les principaux outils utilisés pour les fonctions de learning sont tensorflow et keras.

Les modèles testés sont des versions 2D et 3D de U-Net et certaines de ses variantes.

1.1 Présentation du Dataset

Le dataset est utilisé est le Brain Tumor Segmentation 2020 Dataset (BraTS 2020) [1]. Il fait partie des défis annuels BraTS (Brain Tumor Segmentation Challenge), qui visent à promouvoir le développement de modèles d'apprentissage automatique pour la segmentation des tumeurs cérébrales. Seules les données pour l'entraînement sont disponibles en ligne [2] (celles pour la validation le sont aussi mais sans la vérité terrain, elles sont donc inutilisables dans le cadre d'un apprentissage supervisé). Les images du dataset sont donc utilisées à la fois pour les phases de training, de validation et de test et réparties respectivement selon la proportion 70/15/15 [3].

Le dataset comporte 369 dossiers de patients (numérotés de 001 à 369) contenant chacun des IRM cérébrales 3D de taille 240x240x155. Chaque dossier contient cinq fichiers :

- une IRM pondérée en T1 (fichier *_t1.nii), où la graisse apparaît hyper-intense (couleur claire) et l'eau hypo-intense (couleur sombre)
- une IRM pondérée en T2 (fichier *_t2.nii), où l'eau apparaît hyper-intense et la graisse hypo-intense
- une IRM pondérée en T1 réalisée après l'injection d'un produit de contraste, ici du gadolinium (fichier *_t1ce.nii)
- une IRM FLAIR (fichier *_flair.nii), séquence pondérée en T2 mais avec une suppression du signal du liquide céphalo-rachidien
- un fichier d'annotations contenant les zones segmentées qui indiquent les différentes parties de la tumeur (fichier *_seg.nii)

Tous ces fichiers sont au format NIfTI (*Neuroimaging Informatics Technology Initiative*).

Explication des labels :

- Label 0 (Fond / Tissus sains) : région par défaut, sans anomalie, où il n'y a aucune présence de tumeur ou d'anomalie détectée
- Label 1 (NCR/NET - Noyau de la tumeur nécrotique et non-rehaussée) : partie centrale de la tumeur qui est morte, sans circulation sanguine et qui ne réagit pas à l'agent de contraste (gadolinium)
- Label 2 (ED - Œdème péritumoral) : l'œdème est un gonflement ou une inflammation du tissu cérébral autour de la tumeur. Ce label dénote une réaction des tissus environnants à la tumeur, souvent due à une dysfonction des cellules cérébrales causée par la croissance de la tumeur

- Label 4 (ET - Tumor Enhancing, Tumeur rehaussée par le gadolinium) : cette région est la plus active et visible dans l'IRM après l'injection de gadolinium. Elle représente les zones de tumeur les plus agressives et actives, qui prennent l'agent de contraste

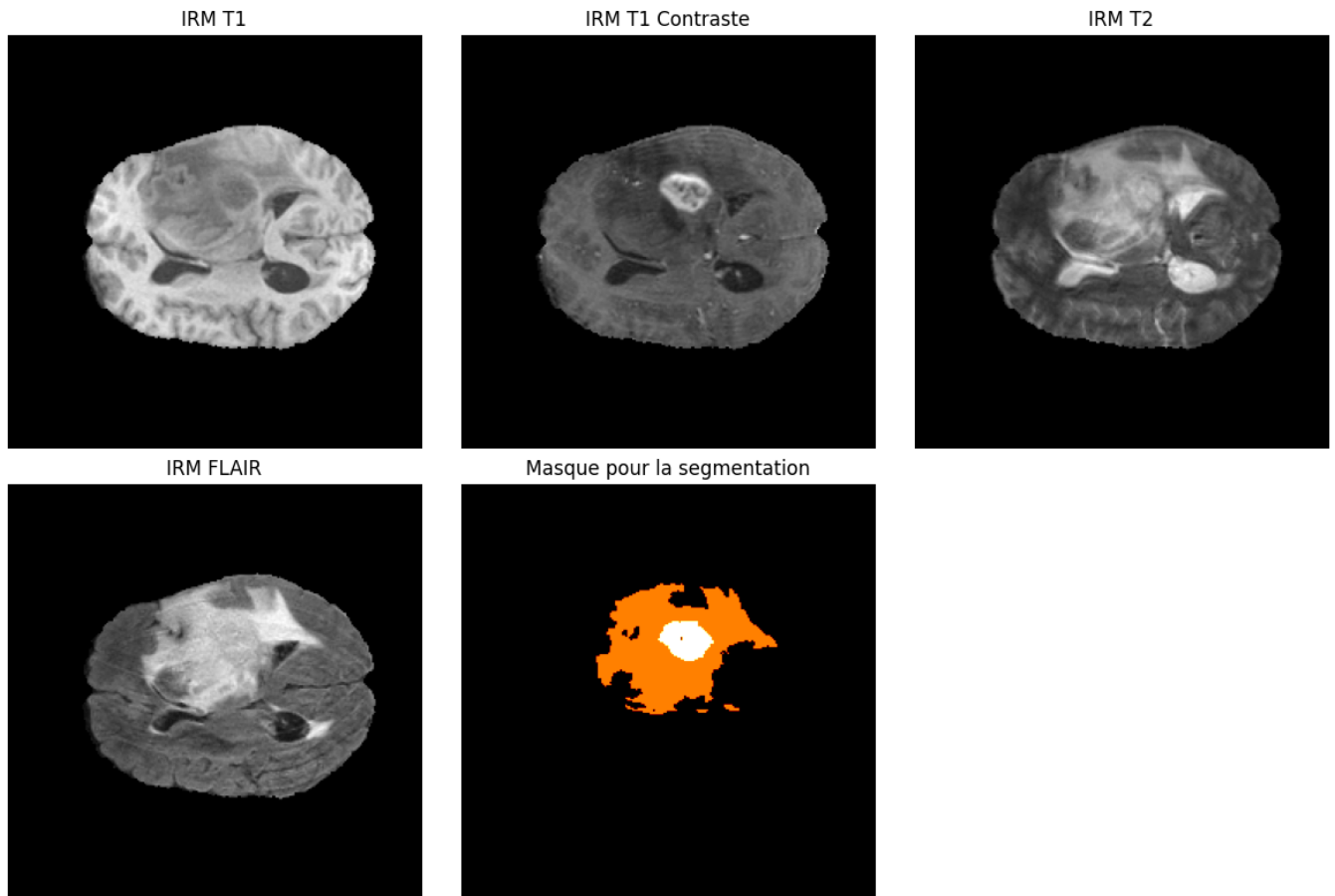


FIGURE 1.1 – Coupes axiales centrales des IRM du patient 001 et masque de la tumeur

1.2 Création du dataset 2D

La première partie du projet utilisant des données 2D, il faut créer un dataset 2D composé d'images extraites des fichiers d'IRM cérébrales 3D [4].

J'ai décidé d'extraire une tranche 2D d'IRM par fichier, c'est à dire quatre par dossier patient (une tranche de l'IRM T1, la tranche correspondante de l'IRM T2 etc) ainsi que la tranche du masque de segmentation associé. J'ai choisi la vue en coupe axiale (aussi appelée coupe sagittale) mais une des deux autres aurait aussi pu faire l'affaire. La tranche sélectionnée selon cette coupe est la médiane car c'est celle qui, en théorie, contient le plus d'information (elle comporte plus de pixels du cerveau qu'une tranche proche des extrémités, sur laquelle la section de l'organe est plus petite). Un exemple du résultat est présenté sur la figure 1.1. Le code permettant d'extraire des tranches d'IRM 3D et de créer le dataset 2D automatiquement est disponible dans le fichier `extracteur.py`.

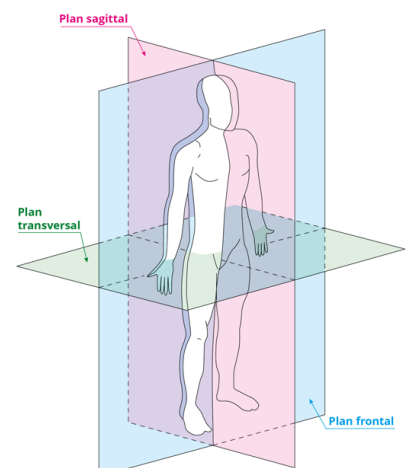


FIGURE 1.2 – Les trois plans du corps

Dans cette première partie les données utilisées sont les images du dataset 2D. Moins volumineuses que les IRM complètes, elles permettent d'effectuer des premiers tests et comparaisons de modèles avec des temps d'entraînements moins importants que pour des données 3D. Les fonctions utilisés pour la phase d'apprentissage sont disponibles dans le fichier `train_2d.py`, tandis que celles utilisées pour la phase de test sont dans le fichier `test_2d.py`.

2.1 Pré-traitement des données

Les données sont pré-traitées de façon identiques pour tous les modèles utilisées dans cette partie.

Les images chargées conservent leur taille de 240 x 240 pixels, qui est acceptable pour entraîner des modèles comme U-Net car elle est assez réduite pour ne pas faire exploser l'utilisation mémoire lors de l'apprentissage (par ailleurs 240 est divisible par $2^4 = 16$, donc cette taille est compatible avec une architecture à 4 niveaux).

Les images sont par contre normalisées pour que les calculs soient ensuite effectués sur des nombres appartenant à la plage $[0,1]$, ce qui accélère et améliore l'entraînement du modèle. Les CNN convergent généralement plus rapidement et de manière plus stable grâce à la normalisation.

Dans le cas où l'on charge plusieurs modalités d'images, ces dernières sont concaténées pour obtenir un tenseur. Celui-ci est de dimension (240,240,4) par exemple si l'on charge les quatre modalités (T1, T2, T1ce et Flair). Ce chargement et pré-traitement est effectué à l'aide de la fonction `load_multimodal_images()` dans le cas de plusieurs modalités.

Les masques de segmentation sont chargés de leur côté et les labels sont encodés au format one-hot.

Enfin, les données d'entraînements (images et masques) sont répartis aléatoirement dans les datasets de training et de validation grâce à la fonction `train_test_split()` de la bibliothèque Scikit-learn. Le paramètre `test_size` (qui correspond à la proportion d'images utilisées pour la validation) est fixé à $0.175 \approx 15/85$ de sorte à ce que 70% des données totales soient utilisées pour la phase de training et 15% pour la phase de validation, comme décidé initialement.

Des méthodes "classiques" de traitement d'images, comme des filtrages ou la méthode d'Otsu par exemple, n'ont pas été utilisées ici mais pourraient être une piste à explorer à l'avenir, pour voir si elles permettent ou non d'améliorer les performances des modèles de deep learning ensuite.

2.2 Modèles

2.2.1 U-Net

Le premier modèle testé est U-Net [5], un CNN de type encodeur-décodeur conçu pour la segmentation d'images, où chaque pixel est classé en fonction de son appartenance à une catégorie/classe. Il se distingue par l'utilisation de "skip connections" entre les couches équivalentes de l'encodeur et du décodeur, qui permettent de préserver la localisation spatiale fine tout en capturant le contexte

global. Bien que développé à l'origine pour des tâches de segmentations d'images médicales, il est aujourd'hui très populaire pour la segmentation dans divers domaines.

L'architecture de ce modèle suit une structure symétrique en U, composée d'un chemin d'encodage (contractant) et d'un chemin de décodage (expansif). Le chemin d'encodage est constitué de plusieurs blocs de convolutions suivis de sous-échantillonnage par max pooling, permettant d'extraire des caractéristiques de plus en plus abstraites tout en réduisant la résolution spatiale. En miroir, le chemin de décodage applique des opérations de transposition de convolution (upsampling) pour reconstruire la résolution d'origine, en combinant à chaque étape les cartes de caractéristiques courantes avec celles de l'encodeur via des connexions de type *skip connections*. Ces connexions facilitent la récupération de l'information spatiale fine qui pourrait être perdue lors de l'encodage.

Enfin, la couche de sortie consiste en une convolution 1×1 avec une activation softmax, appliquée canal par canal, afin de produire une probabilité pour chaque classe par pixel. Ce choix est adapté à un problème de segmentation multi-classe.

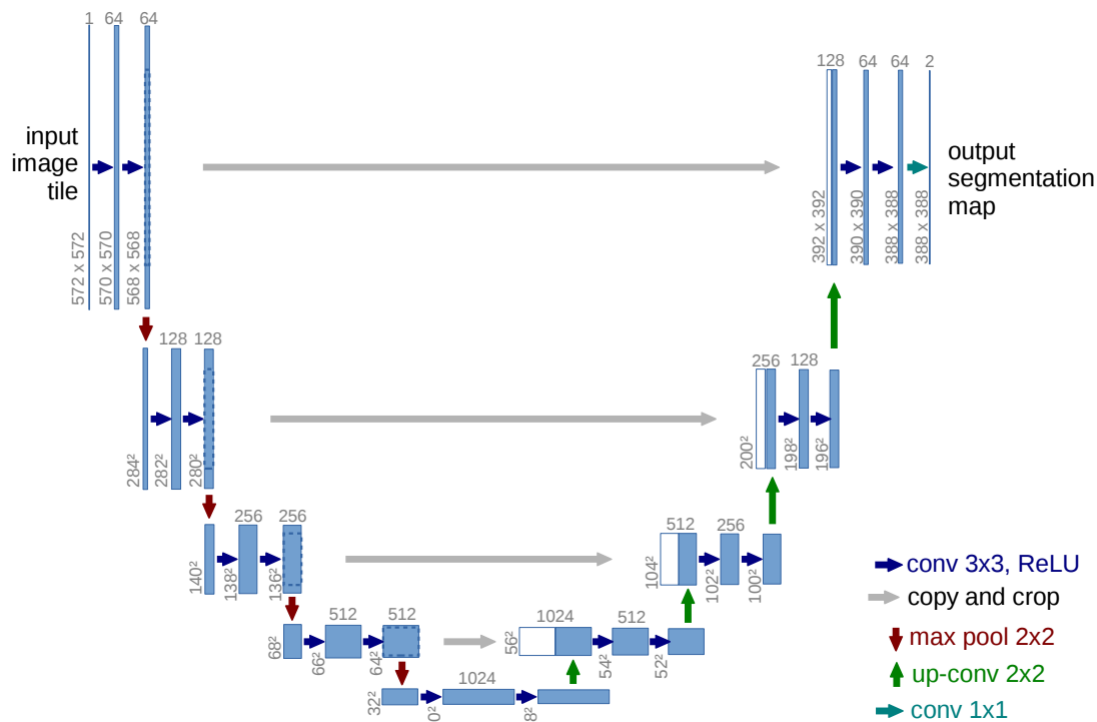


FIGURE 2.1 – Architecture de U-Net

Dans mon implémentation, j'ai testé quelques variantes de l'architecture, l'une avec un goulot d'étranglement (*bottleneck*) de 512 filtres et un autre de 1024 filtres, afin d'évaluer l'impact de ce paramètre les performances de segmentation. De plus, l'effet de la "normalisation de lot" (*BatchNormalization*) a été étudié. Cette dernière, insérée après chaque couche convolutionnelle, permet de stabiliser et accélérer l'apprentissage en réduisant la covariate shift interne. Toutefois, elle peut aussi affecter la dynamique du modèle lors de l'inférence si elle n'est pas correctement gérée (par exemple, lors de la prédiction sur des échantillons isolés). L'effet de l'ajout de couches "d'abandon" (dropout) dans le bottleneck a également été testé. Son but de réduire le surapprentissage en introduisant une régularisation stochastique pendant l'entraînement.

L'optimiseur Adam est utilisé pour mettre à jour les poids du modèle, et "Categorical Crossentropy" est la fonction de perte utilisée pour l'entraînement car les labels des classes sont des vecteurs binaires (encodage one-hot) et non des nombres entiers.

Une fois le modèle entraîné on peut le sauvegarder lui et les poids avec la fonction `model.save()`, et le recharger plus tard avec `keras.models.load_model()`.

2.2.2 Attention U-Net

L'Attention U-Net [6] est une extension du U-Net qui incorpore des modules d'attention dans les connexions de saut (skip connections) afin de pondérer dynamiquement les caractéristiques spatiales importantes. Ces modules permettent en théorie au réseau de filtrer les informations transmises du chemin encodeur vers le décodeur, en mettant l'accent sur les régions pertinentes pour la tâche de segmentation, ce qui améliore la performance, en particulier sur des structures fines ou faiblement contrastées.

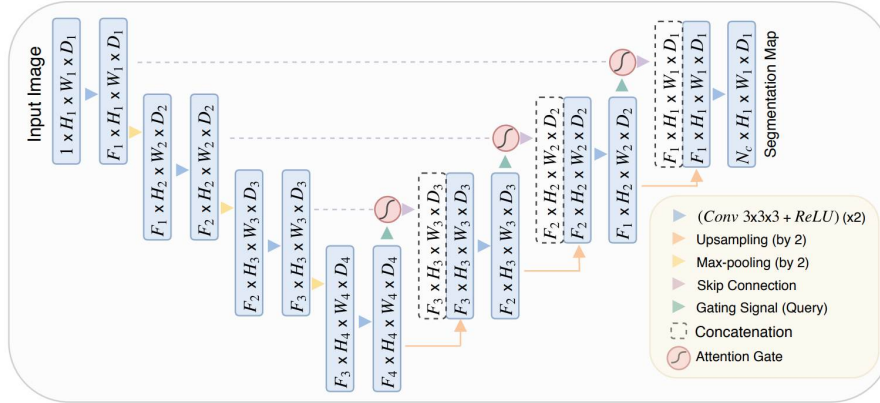


FIGURE 2.2 – Architecture d'Attention U-Net

2.2.3 U-Net++

U-Net++ [7] est une amélioration architecturale de U-Net, conçue pour améliorer la précision de la segmentation. Son but est vise à réduire le fossé sémantique entre les caractéristiques extraites dans le chemin d'encodage et celles du décodage. Pour ça, U-net++ introduit une série de sous-blocs de convolution entre les couches correspondantes de l'encodeur et du décodeur, formant une structure dense de type grille. Cela permet une agrégation progressive et hiérarchique des caractéristiques à différentes résolutions et améliore la capacité du réseau à capturer à la fois des détails fins et des contextes larges.

Dans la littérature on trouve plusieurs cas où U-Net++ a montré de meilleures performances que U-Net classique sur des tâches de segmentation, notamment dans le domaine biomédical, d'où l'envie de le tester dans le cadre de ce projet.

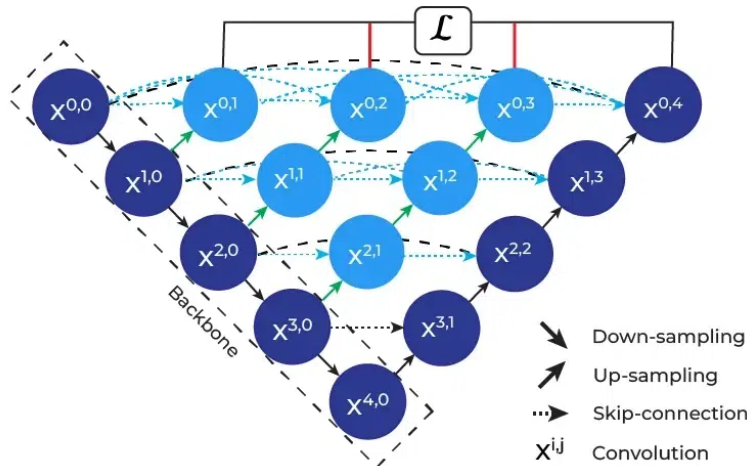


FIGURE 2.3 – Architecture de U-Net++

2.3 Métriques d'entraînement

Pendant l'entraînement il est intéressant d'enregistrer des métriques telles que la précision (ou accuracy, qui est la proportion de prédictions correctes par rapport au nombre total de prédictions) et les pertes (ou loss, qui mesurent à quel point les prédictions du modèle sont éloignées des vraies valeurs), à la fois pour le set d'entraînement et de validation. Ces données sont stockées dans l'objet `history` retourné par la fonction `model.fit()`. On peut ensuite les extraire pour visualiser les courbes d'accuracy et de loss avec matplotlib.

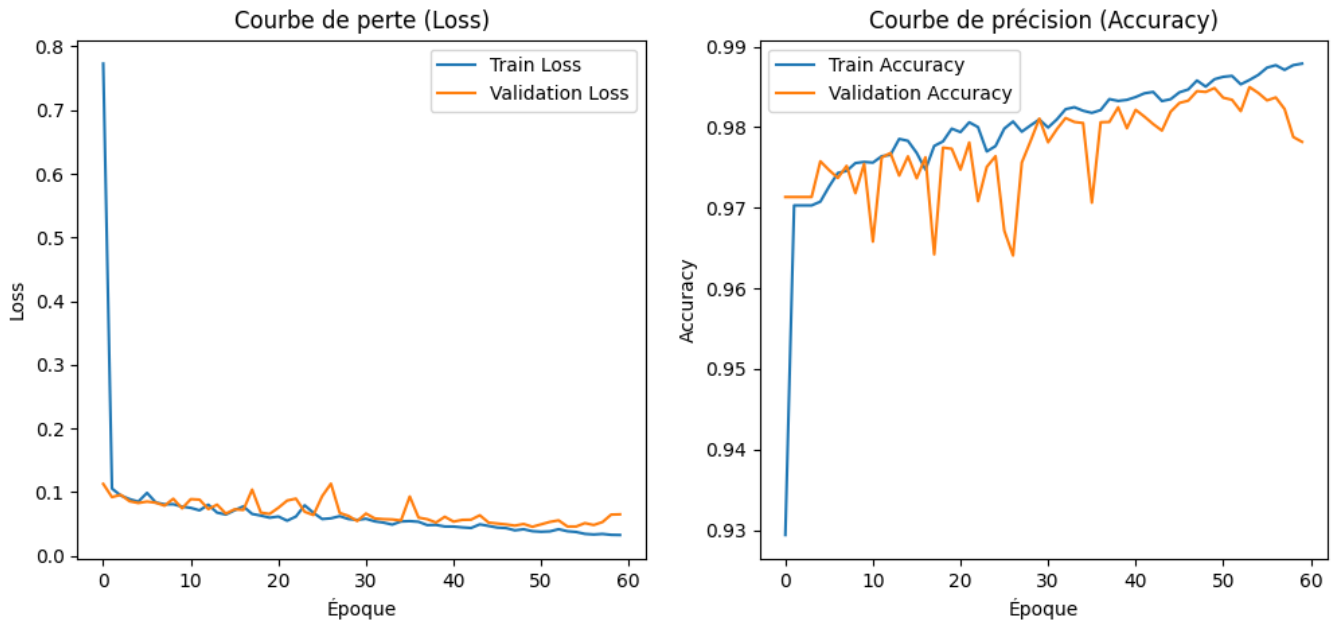


FIGURE 2.4 – Graphiques avec les courbes de pertes (à gauche) et de précisions (à droite)

Ces données sont importantes pour évaluer le modèle et trouver d'éventuels problèmes. Par exemple une précision d'entraînement forte mais accompagnée d'une précision de validation qui stagne et n'arrive pas aux mêmes valeurs peut indiquer un surentraînement.

Par ailleurs, grâce à la fonction `EarlyStopping()` de Keras, on peut imposer un arrêt prématuré de l'entraînement si une certaine métrique ne s'améliore pas pendant un certain nombre d'epochs. Ici j'ai décidé que l'arrêt serait déclenché si la perte de validation (*Validation loss*) ne diminuait pas pendant plus de 10 epochs. Ce choix est motivé par l'objectif de minimiser l'erreur de segmentation sur les données de validation, et `val_accuracy` n'est pas fiable en segmentation multi-classes, car une forte classe dominante (par exemple les pixels du fond des IRM labellisés "0" rendent cette classe bien plus représentée que les trois autres) peut donner une fausse impression de bonne performance.

En outre, la méthode `time.time()` de Python permet de calculer et retourner le temps d'entraînement d'un modèle (en secondes).

```
Epoch 85: val_loss did not improve from 0.05478
17/17 ————— 8s 478ms/step - accuracy: 0.9827 - loss: 0.0483 - val_accuracy: 0.9817 - val_loss: 0.055
Epoch 85: early stopping
Temps d'entraînement : 776.97 secondes
```

FIGURE 2.5 – Exemple d'affichage des valeurs finales des précisions, des pertes et du temps d'entraînement d'un modèle

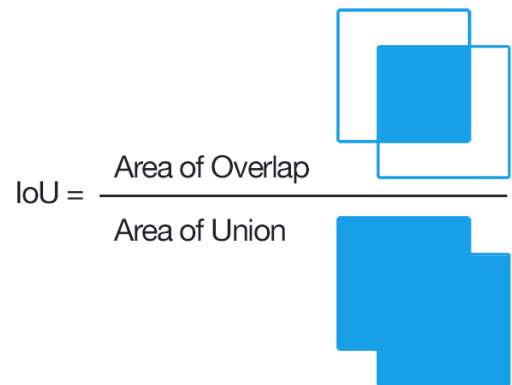
2.4 Métriques de test

2.4.1 Mesures de performance

Pour évaluer la qualité de la segmentation produite par un modèle, j'utilise deux métriques couramment employées dans les tâches de segmentation d'image : l'Intersection over Union (IoU) et le Dice Similarity Coefficient (DSC), également appelé score de Dice.

Intersection over Union (IoU) Le score IoU mesure le recouvrement entre la zone prédite par le modèle et la zone réellement annotée (vérité terrain). Il est défini comme le rapport entre l'intersection et l'union des deux ensembles de pixels :

$$\text{IoU} = \frac{|\text{Prédiction} \cap \text{Vérité terrain}|}{|\text{Prédiction} \cup \text{Vérité terrain}|}$$

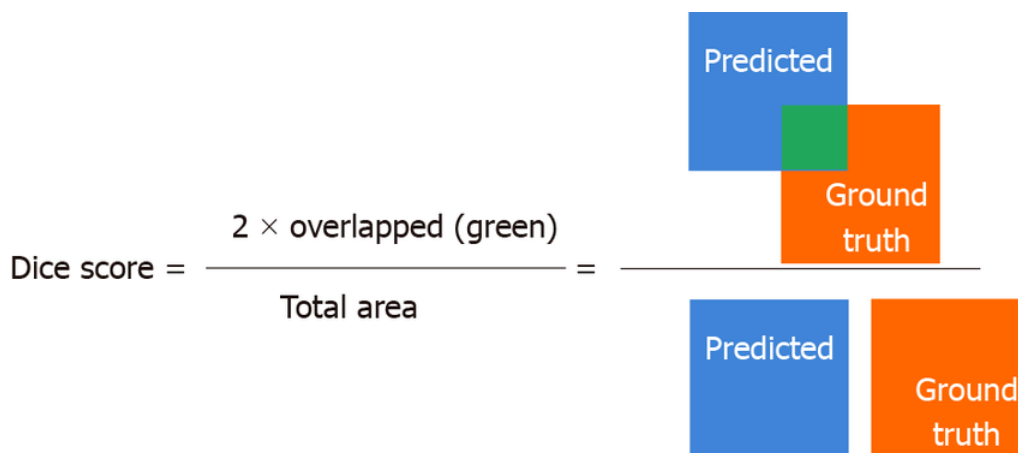


Ce score varie entre 0 et 1, où 1 indique une correspondance parfaite entre la prédiction et la vérité terrain. Une IoU proche de 0 traduit un faible recouvrement.

Score de Dice Le score de Dice, similaire au score IoU, est une autre métrique de similarité, mais il donne un poids plus important aux zones d'intersection. Il est défini comme suit :

$$\text{Dice} = \frac{2 \times |\text{Prédiction} \cap \text{Vérité terrain}|}{|\text{Prédiction}| + |\text{Vérité terrain}|}$$

Le score de Dice prend également des valeurs entre 0 et 1. Il est particulièrement utile dans les contextes médicaux où les classes d'intérêt (par exemple, les tumeurs) sont souvent de petite taille par rapport à l'image globale, car il est plus sensible aux faux négatifs que l'IoU.



Utilisation dans le projet Pour chaque test de modèle, ces deux scores sont calculés pour chacune des quatre classes de segmentation et la moyenne est ensuite faite pour obtenir une mesure globale de performance. Cela permet d'évaluer à la fois la précision de la segmentation et la capacité du modèle à détecter correctement toutes les régions pertinentes, même celles de petite taille.

```
IoU moyen: 0.5220801643256849
IoU par classe: [0.9905566873982187, 0.22649869214754711, 0.4909453609122549, 0.38031991684471905]
Coefficient de Dice moyen: 0.6435569052952179
Coefficient de Dice par classe: [0.9952559438967175, 0.36934192200557103, 0.658569218944232, 0.5510605363343514]
```

FIGURE 2.6 – Exemple d’affichage des scores IoU et Dice pour les différentes classes prédites sur l’ensemble des images de test par un modèle, ainsi que leurs moyennes

2.4.2 Matrice de confusion

La matrice de confusion est un outil très utile pour évaluer les performances d’un modèle de classification. Il s’agit d’un tableau carré dont chaque ligne correspond aux pixels appartenant à une classe réelle (vérité terrain), tandis que chaque colonne correspond aux pixels prédits par le modèle pour cette même classe. Chaque case indique donc le nombre de pixels pour lesquels la prédiction et la vérité terrain coïncident ou divergent.

Par exemple, la diagonale principale représente les pixels correctement classifiés, tandis que les cases hors diagonale indiquent les erreurs de classification (pixels mal assignés à une classe différente).

Dans le cadre de la segmentation des tumeurs cérébrales, cette matrice permet non seulement d’identifier la performance globale du modèle, mais aussi de visualiser précisément quelles classes sont souvent confondues entre elles, ce qui est crucial pour améliorer la qualité de la segmentation.

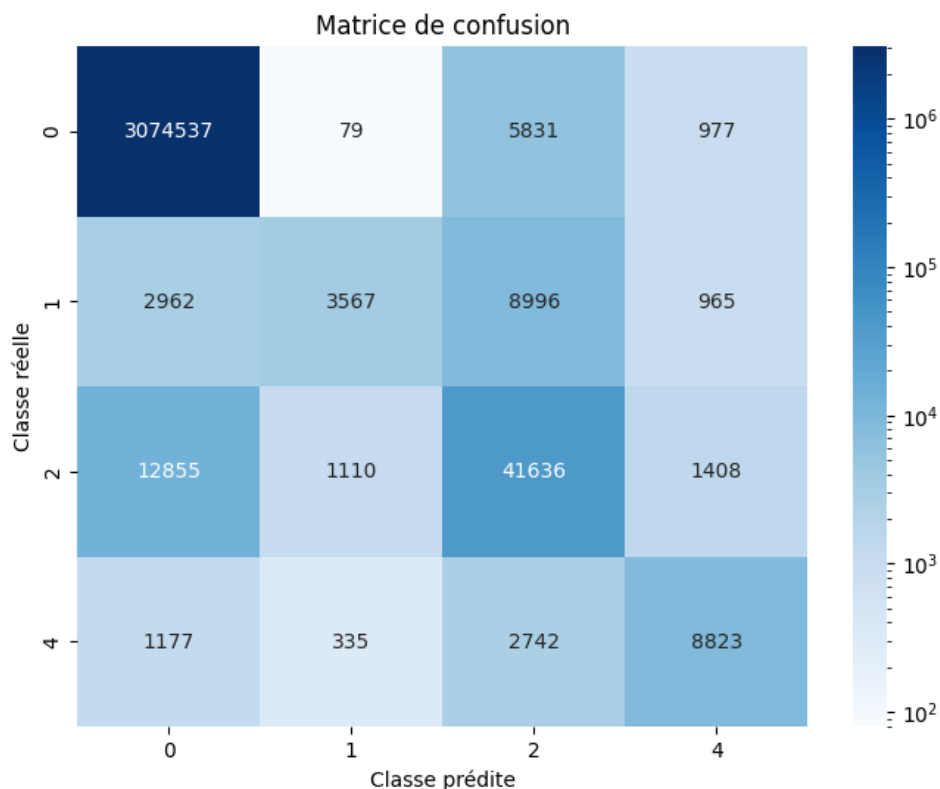


FIGURE 2.7 – Exemple de matrice de confusion des pixels pour les 4 classes de segmentation sur l’ensemble du dataset de test (55 patients)

Une échelle logarithmique est utilisée pour les couleurs de la matrice de confusion, sinon toutes les cases à l'exception de 0/0 seraient blanches, à cause du nombre bien plus important (au moins 10^3 fois plus) de pixels dans cette catégorie (Fond et tissus sains) que dans les autres.

2.4.3 Affichage de la prédiction

Enfin, il est bien sûr aussi possible d'afficher les zones qui correspondent aux différentes classes prédites pour une image en particulier, et de les comparer avec le masque de segmentation (vérité terrain) du même patient.

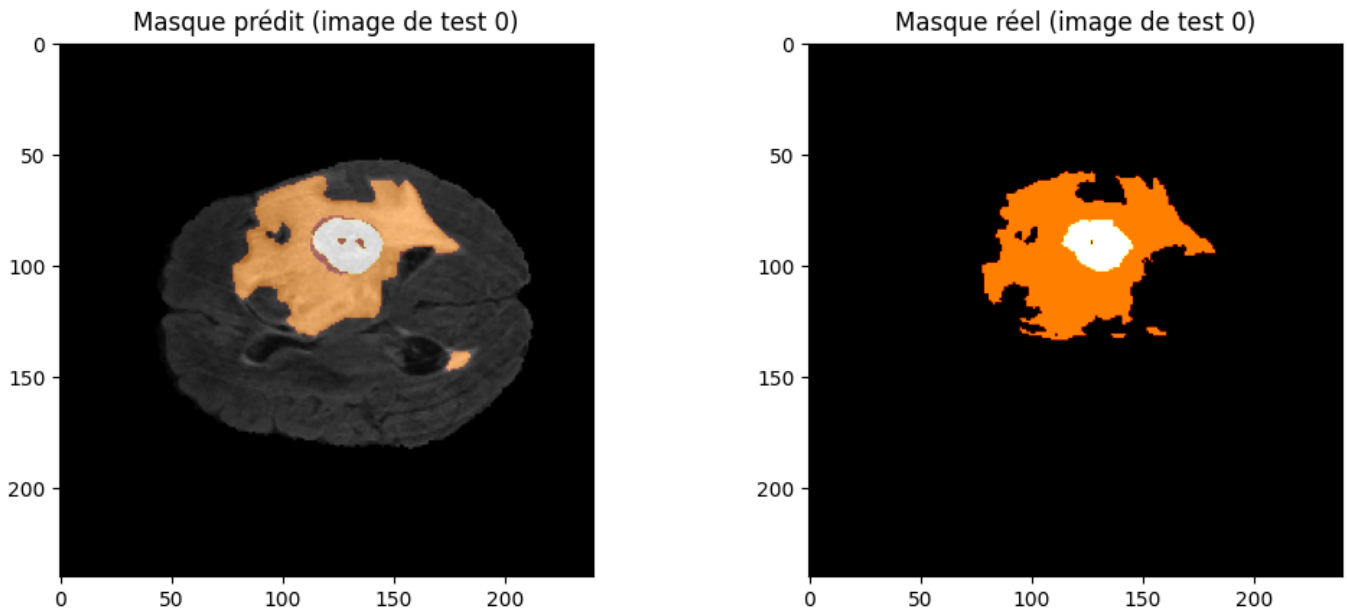


FIGURE 2.8 – *Comparaison des masques prédit et réel pour l'image de test 0, qui correspond au dossier patient 001. Le modèle utilisé est U-Net++ avec un bottleneck de 1024 filtres et l'utilisation de couches de BatchNormalization*

Avec la palette de couleurs *afmhot* de matplotlib utilisée, les labels apparaissent avec les couleurs suivantes sur les masques : label 0 en noir, label 1 en rouge, label 2 en orange et label 4 et blanc.

2.5 Résultats

2.5.1 Comparaison de l'entraînement avec une seule et quatre modalités

Dans un premier temps on compare les performances de deux modèles U-Net identiques mais l'un entraîné uniquement sur les IRM de type Flair ("U-Net Flair") et l'autre entraîné avec les quatre modalités disponibles ("U-Net All").

TABLE 2.1 – Comparaison complète des métriques d'entraînements et performances des deux modèles

Métrique	U-Net Flair		U-Net All	
Accuracy (train)	0.9827		0.9836	
Loss (train)	0.0483		0.0486	
Accuracy (val)	0.9817		0.9775	
Loss (val)	0.0553		0.0654	
Temps d'entraînement	329 s		351 s	
Classe	IoU	Dice	IoU	Dice
Classe 0 (Fond + Tissus sains)	0.99	0.99	0.99	0.99
Classe 1 (Noyau tumoral)	0.12	0.21	0.21	0.34
Classe 2 (Œdème péritumoral)	0.51	0.68	0.40	0.58
Classe 4 (Tumeur rehaussée)	0.04	0.07	0.22	0.36
Moyenne segmentation	0.41	0.49	0.45	0.57

Comme on pouvait s'y attendre, la version entraînée avec toutes les modalités donne de meilleurs résultats (scores moyens IoU et Dice plus élevés) mais le temps d'entraînement est un peu plus long qu'avec une seule modalité.

- [1] CBICA. *Multimodal Brain Tumor Segmentation Challenge 2020 : Data*. Consulté le 23 février 2025. URL : <https://www.med.upenn.edu/cbica/brats2020/data.html>.
- [2] AWSAF. *BraTS2020 Dataset (Training + Validation)*. Consulté le 23 février 2025. 2020. URL : <https://www.kaggle.com/datasets/awsaf49/brats20-dataset-training-validation>.
- [3] VILLETTE. *BraTS2020 3D*. <https://www.kaggle.com/datasets/lonidasspartiate/brats2020-3d>. Accessed : 2025-05-01. 2025.
- [4] VILLETTE. *BraTS2020 2D*. <https://www.kaggle.com/datasets/lonidasspartiate/2d-brain>. Accessed : 2025-05-01. 2025.
- [5] Olaf RONNEBERGER, Philipp FISCHER et Thomas BROX. *U-Net : Convolutional Networks for Biomedical Image Segmentation*. 2015. arXiv : 1505.04597 [cs.CV]. URL : <https://arxiv.org/abs/1505.04597>.
- [6] Ozan OKTAY et al. *Attention U-Net : Learning Where to Look for the Pancreas*. 2018. arXiv : 1804.03999 [cs.CV]. URL : <https://arxiv.org/abs/1804.03999>.
- [7] Zongwei ZHOU et al. *UNet++ : A Nested U-Net Architecture for Medical Image Segmentation*. 2018. arXiv : 1807.10165 [cs.CV]. URL : <https://arxiv.org/abs/1807.10165>.